```
  gem "rack"
  gem "rackup"
end

class PrefixingMiddleware
  def initialize(app)
    @app = app
  end

  def call(env)
    status, headers, body = @app.call(env)
    new_body = ["<h2>This is a prefix</h2>"] + body
    [status, headers, new_body]
  end
end

class PostfixingMiddleware
  def initialize(app)
    @app = app
  end

  def call(env)
    status, headers, body = @app.call(env)
    new_body = body + ["<h2>This is a postfix</h2>"]
    [status, headers, new_body]
  end
end

class RackApplication
  def call(env)
    [
      200,
      {"content-type" => "text/html"},
      ["<h1>Welcome to Rack</h1>"]
    ]
  end
end

use PrefixingMiddleware
use PostfixingMiddleware
run RackApplication.new
```

This Rack application is built from three smaller applications, PrefixingMiddleware, PostfixingMiddleware, and RackApplication. Where our previous example just used the run command, this example has the use command once for each of the middleware classes. The use command takes as an argument the name of the middleware class, in contrast to the run method, which takes an instance of a Rack app as its argument.

When you execute this file with rackup, the following things happen:

- Rack takes the instance passed to run as the first application.
- Then starting from the bottommost use call and working up, Rack creates an instance of each middleware class, passing it the previous instance as the argument. In our case, an instance of PostfixingMiddleware is created with the RackApplication instance as the argument, and then an instance of PrefixingMiddleware is created with that previous instance as the argument.